EXERCÍCIO INTEGRADOR — MINI HELP DESK (MVC + OO + Singleton)

Objetivo

Construir um mini sistema inspirado no *Helpdesk Pro*, aplicando:

- ✅ Orientação a Objetos (classes, encapsulamento, construtor)
- ✅ Padrão Repository (interface + implementação)
- ✅ Arquitetura MVC (Controller + View simples)
- ✅ PDO com prepared statements
- ✅ Padrão Singleton (conexão com banco)

CENÁRIO DO PROBLEMA

Você deve desenvolver um sistema simples de **cadastro de chamados (tickets)**.

O sistema permitirá:

- Criar um chamado
- Listar chamados
- Buscar chamado por ID

ESTRUTURA ESPERADA (baseada nas aulas)

Organize seu projeto assim:

```
src/
├── Domain/
│   ├── Entidade/
│   │   └── Chamado.php
│   └── Repositorio/
│       └── ChamadoRepositorioInterface.php
│
├── Infra/
│   ├── Database/
│   │   └── Database.php   (Singleton)
│   └── Repositorio/
│       └── ChamadoRepositorioPDO.php
│
├── Controller/
```

```
│   └── ChamadoController.php
│
└── Core/
    ├── Request.php
    ├── Response.php
    └── View.php
```

Baseado diretamente na separação Domain / Infra / Controller vista nas aulas

---

PARTE 1 — ENTIDADE (OO + Encapsulamento)

Crie a classe: Chamado

Requisitos:

1. Atributos privados:
    - id
    - titulo
    - descricao
    - status
2. O construtor deve receber:
    - titulo
    - descricao
    - status padrão = "aberto"
3. Crie getters para todos os atributos
4. Crie um método:

public function alterarStatus(string $novoStatus): void

Validação:

- Só permitir:
    - aberto
    - em_andamento
    - resolvido

Dica: igual ao conceito de encapsulamento visto no material — nunca deixe alterar direto

---

PARTE 2 — INTERFACE REPOSITORY

Crie:

ChamadoRepositorioInterface

Métodos obrigatórios:

public function salvar(Chamado $chamado): void;

public function listar(): array;

public function buscarPorId(int $id): ?Chamado;

Aqui você está aplicando o padrão Repository (separar regra de negócio do banco)

PARTE 3 — SINGLETON + PDO

Crie a classe Database (Singleton)

Requisitos:

1. A classe deve ter:
    - propriedade estática privada para guardar a instância
    - construtor privado
2. Método principal:

public static function getInstance(): PDO

3. Configurar PDO com:

- charset utf8mb4
- ERRMODE_EXCEPTION
- prepared statements

Baseado na aula de PDO + Singleton

PARTE 4 — REPOSITÓRIO COM PDO

Crie:

ChamadoRepositorioPDO

Requisitos:

- Implementar a interface criada
- Usar o Database (Singleton)

🔹 **Método salvar()**

- Inserir no banco usando **prepared statement**
- NÃO concatenar SQL (evitar SQL Injection)

🔹 **Método listar()**

- Retornar array de objetos Chamado

🔹 **Método buscarPorId()**

- Retornar objeto ou null

Dica importante da aula:

Nunca misturar SQL com lógica de negócio diretamente

PARTE 5 — CONTROLLER (MVC)

Crie:

ChamadoController

Métodos:

```
public function index(): void
public function criar(): void
public function salvar(): void
public function exibir(int $id): void
```

🔹 **index()**

- Lista todos os chamados

🔹 **criar()**

- Exibe formulário simples (pode ser echo mesmo)

🔹 **salvar()**

- Recebe dados do Request
- Cria objeto Chamado
- Chama repositório

🔹 **exibir()**

- Mostra dados de um chamado específico

Fluxo baseado na aula de MVC (Request → Controller → View)

PARTE 6 — VIEW (SIMPLES)

Você pode fazer algo simples como:

echo "Lista de chamados:";

Ou separar em arquivos .php (melhor).

🧪 **PARTE 7 — SIMULAÇÃO (IMPORTANTE)**

Crie um arquivo tipo:

public/index.php

Simule:

- Criar 2 chamados
- Listar todos
- Buscar por ID

REGRAS IMPORTANTES (AVALIAÇÃO)

Você **NÃO pode**:

❌ Acessar atributos diretamente
❌ Usar SQL concatenado
❌ Criar várias conexões com banco
❌ Misturar Controller com SQL

Você **DEVE**:

✅ Usar encapsulamento
✅ Usar interface
✅ Usar Singleton
✅ Usar prepared statements

DICAS (SEM ENTREGAR RESPOSTA)

- Pense:
  "Quem é responsável por isso?"
  (Controller? Repository? Entidade?)
- Se você está escrevendo SQL fora do repositório → está errado
- Se você usa new PDO em vários lugares → faltou Singleton
- Se consegue mudar atributos direto → faltou encapsulamento

DESAFIO EXTRA (SE QUISER SUBIR NÍVEL)

- Adicionar:
  - atualização de status
  - deletar chamado
  - paginação simples

RESULTADO ESPERADO

Ao final, você terá:

✓ Um mini sistema MVC funcionando
✓ Código organizado em camadas
✓ Uso real de PDO + Singleton
✓ Base pronta para evoluir para projeto grande